“Compiler Design

Unit 4

Intermediate Re presentation

Available on notes.arpanadhikari7.com.np

Introduction

- Compilers are typically organized as a series of passes.
- As the compiler derives knowledge about the code it compiles, it must convey that information from one pass to another.
- Thus, the compiler needs a representation for all of the facts that it derives about the program. We call this representation an *intermediate representation, or IR.*
- An **Intermediate Representation (IR)** is a structured, machine-independent form of code used by compilers to represent source programs between the **front-end** (parsing, semantic analysis) and the **back-end** (optimization, code generation).
- It bridges high-level source code and low-level machine code.

Available on notes.arpanadhikari7.com.np

Why Use an Intermediate Representation?

- **Machine Independence**: Allows the same IR to be used for different target architectures.
- **Optimization**: Enables machine-independent optimizations before code generation.
- **Modularity**: Separates front-end (language-specific) and back-end (target-specific) phases.
- **Reusability**: Multiple front-ends (C, Java, Python) can share the same IR and back-end.

Available on notes.arpanadhikari7.com.np

A Taxonomy of Intermediate Representations

- Compilers have used many kinds of IR.
- The IR can be organized as structural organization, level of abstraction, and naming discipline.
- In general, these three attributes are independent; most combinations of organization, abstraction, and naming have been used in some compiler.

Available on notes.arpanadhikari7.com.np

- Broadly speaking, IRs fall into three structural categories:
- **Graphical IRs** encode the compiler's knowledge in a graph.
- The algorithms are expressed in terms of graphical objects: nodes, edges, lists, or trees.
- Represent programs as trees or graphs, preserving high-level semantics.
- Abstract Syntax Tree (AST), Directed Acyclic Graph (DAG), Control Flow Graph (CFG) etc belongs to graphical IR

- **Linear IRs** resemble pseudo-code for some abstract machine. The algorithms iterate over simple, linear sequences of operations.
- Three-Address Code (TAC), Stack Machine Code, Register Transfer Language (RTL) belongs to the linear IRS

Available on notes.arpanadhikari7.com.np

- Hybrid IRs combine elements of both graphical and linear IRs, in an attempt to capture their strengths and avoid their weaknesses.

- A common hybrid representation uses a low-level linear IR to represent blocks of straight-line code and a graph to represent the flow of control among those blocks.
- **LLVM (Low-Level Virtual Machine) IR, GIMPLE (GCC) etc belongs to hybrid Irs.**

- GIMPLE is a **simplified, three-address code (TAC) representation** used in the **GNU Compiler Collection (GCC)** for mid-level optimizations. It bridges the gap between **high-level ASTs** (like GENERIC) and **low-level RTL** (Register Transfer Language).

Available on notes.arpanadhikari7.com.np

- The structural organization of an IR has a strong impact on how the compiler writer thinks about analysis, optimization, and code generation.
- For example, tree like IRs lead naturally to passes structured as some form of tree walk.
- Similarly, linear IRs lead naturally to passes that iterate over the operations in order.

Available on notes.arpanadhikari7.com.np

- The second axis of IR taxonomy is the level of abstraction at which the IR represents operations.

- The IR can range from a near-source representation in which a single node might represent an array access or a procedure call to a low-level representation in which several IR operations must be combined to form a single target-machine operation.

Available on notes.arpanadhikari7.com.np

- The third axis of our IR taxonomy deals with the name space used to represent values in the code.
- In translating source code to a lower-level form, the compiler must choose names for a variety of distinct values.
- For example, to evaluate a - 2 × b in a low-level IR, the compiler might generate a sequence of operations.

  t1 → b

  t2 →2 × t1

  t3 → a

  t4 → t3 - t2

Available on notes.arpanadhikari7.com.np

Graphical IRs

- In **compiler design**, a **graphical intermediate representation (IR)** is a **visual or graph-based form** of representing the structure of source code during compilation.
- It serves as a bridge between the high-level source code and the low-level machine code, allowing optimizations and transformations to occur more easily.
- Many compilers use IRs that represent the underlying code as a graph.
- While all the graphical IRs consist of nodes and edges, they differ in their level of abstraction, in the relationship between the graph and the underlying code, and in the structure of the graph.

Available on notes.arpanadhikari7.com.np

Common Graphical Intermediate Representations:

Syntax related Trees

- The parse trees are graphs that represent the source code form of the program.
- Parse trees are one specific form of treelike IRs.
- In most treelike IRs, the structure of the tree corresponds to the syntax of the source code.

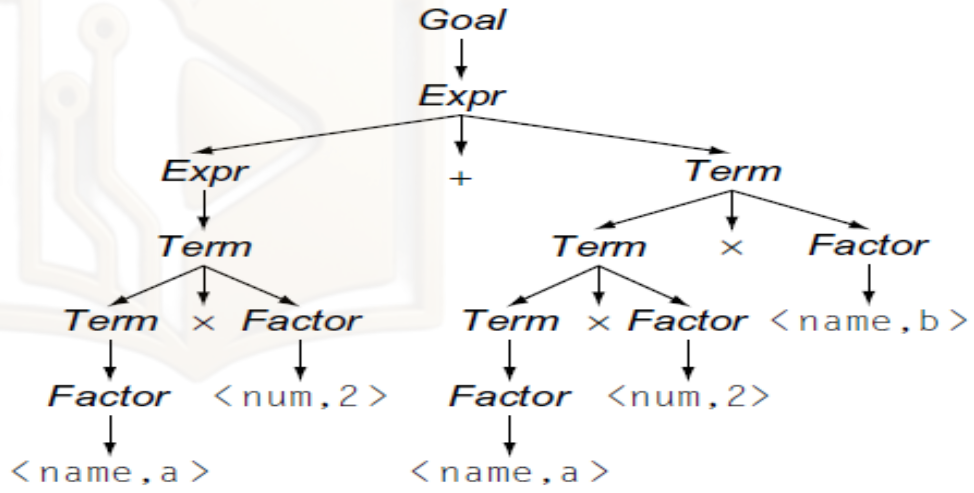
Available on notes.arpanadhikari7.com.np

Parse tree

- the *parse tree is a graphical representation*
- for the derivation, or parse, that corresponds to the input program.

Goal	→	*Expr*
Expr	→	*Expr* + *Term*
	|	*Expr* - *Term*
	|	*Term*
Term	→	*Term* × *Factor*
	|	*Term* ÷ *Factor*
	|	*Factor*
Factor	→	(*Expr*)
	|	num
	|	name

a) Classic Expression Grammar



(b) Parse Tree for a × 2 + a × 2 × b

FIGURE Parse Tree for a × 2 + a × 2 × b Using the Classic Expression Grammar.

Available on notes.arpanadhikari7.com.np

- Figure above shows the classic expression grammar alongside a parse tree for a × 2 + a × 2 × b.

- The parse tree is large relative to the source text because it represents the complete derivation, with a node for each grammar symbol in the derivation.

- Since the compiler must allocate memory for each node and each edge, and it must traverse all those nodes and edges during compilation.

Available on notes.arpanadhikari7.com.np

- Needs to think a way to shrink this parse tree.
- An effective technique is to abstract away those nodes that serve no real purpose in the rest of the compiler.
- This approach leads to a simplified version of the parse tree, called an abstract syntax tree.

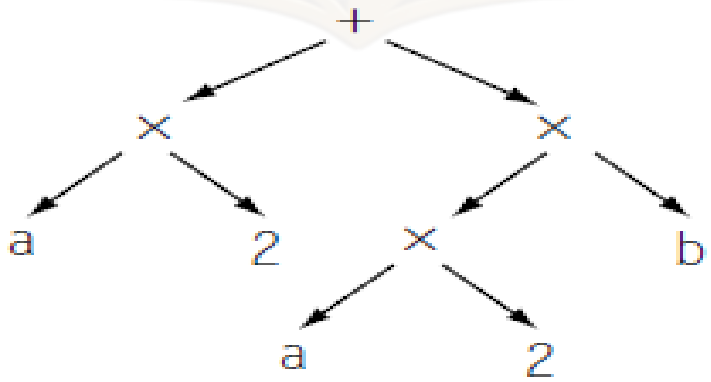
Abstract Syntax Trees

- An AST is a contraction of the parse tree that omits most nodes for non terminal symbols.

Available on notes.arpanadhikari7.com.np

- The abstract syntax tree (AST) retains the essential structure of the parse tree but eliminates the extraneous nodes.

- The precedence and meaning of the expression remain, but extraneous nodes have disappeared. Here is the AST for a × 2 + a × 2 × b:



Available on notes.arpanadhikari7.com.np

- Abstract syntax trees have found widespread use.
- Many compilers and interpreters use them; the level of abstraction that those systems need varies widely.
- If the compiler generates source code as its output, the AST typically has source-level abstractions.
- If the compiler generates assembly code the final version of the AST is usually at or below the abstraction level of the machine's instruction set.

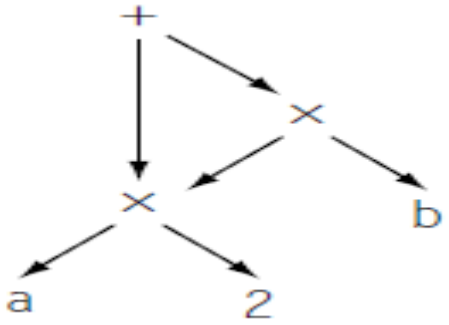
Available on notes.arpanadhikari7.com.np

Directed Acyclic Graphs

- A DAG is an AST with sharing. Identical sub-trees are instantiated once, with multiple parents.
- While the AST is more concise than a syntax tree, it faithfully retains the structure of the original source code.
- For example, the AST for a × 2 + a × 2 × b contains two distinct copies of the expression a × 2.
- A directed acyclic graph (DAG) is a contraction of the AST that avoids this duplication.
- In a dag, nodes can have multiple parents, and identical sub-trees are reused.
- Such sharing makes the DAG more compact than the corresponding AST.

Available on notes.arpanadhikari7.com.np

- The DAG for a × 2 + a × 2 × b, shown below, reflects this fact by sharing a single copy of a × 2.
- The DAG encodes an explicit hint for evaluating the expression.
- If the value of a cannot change between the two uses of a, then the compiler should generate code to evaluate a × 2 once and use the result twice.



Available on notes.arpanadhikari7.com.np

- If the value of a cannot change between the two uses of a, then the compiler should generate code to evaluate a × 2 once and use the result twice.
- This strategy can reduce the cost of evaluation.
- However, the compiler must prove that a's value cannot change.
- If the expression contains neither assignment nor calls to other procedures, the proof is easy.
- Since an assignment or a procedure call can change the value associated with a name, the DAG construction algorithm must invalidate subtrees as the values of their operands change.

Available on notes.arpanadhikari7.com.np

- DAGs are used in real systems for two reasons.
- If memory constraints limit the size of programs that the compiler can handle, using a DAG can help by reducing the memory footprint.
- Other systems use DAGs to expose redundancies.
- Here, the benefit lies in better compiled code.

Available on notes.arpanadhikari7.com.np

Advantages

- sharing of values is explicit
- exposes redundancy (value computed twice)

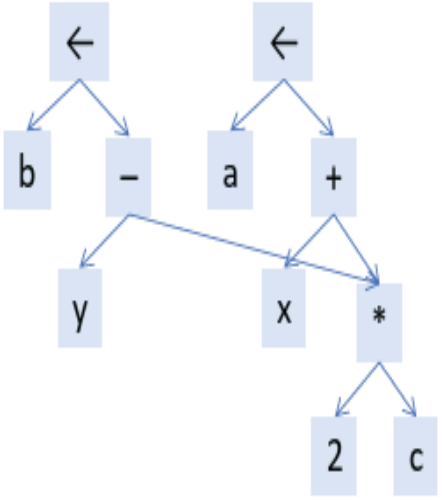
Disadvantages

- difficult to transform (e.g., delete a stmt)
- not useful for showing control flow structure

⇒ Better for *analysis* than *transformation*

Example

a ← x + 2 * c;
b ← y – 2 * c;

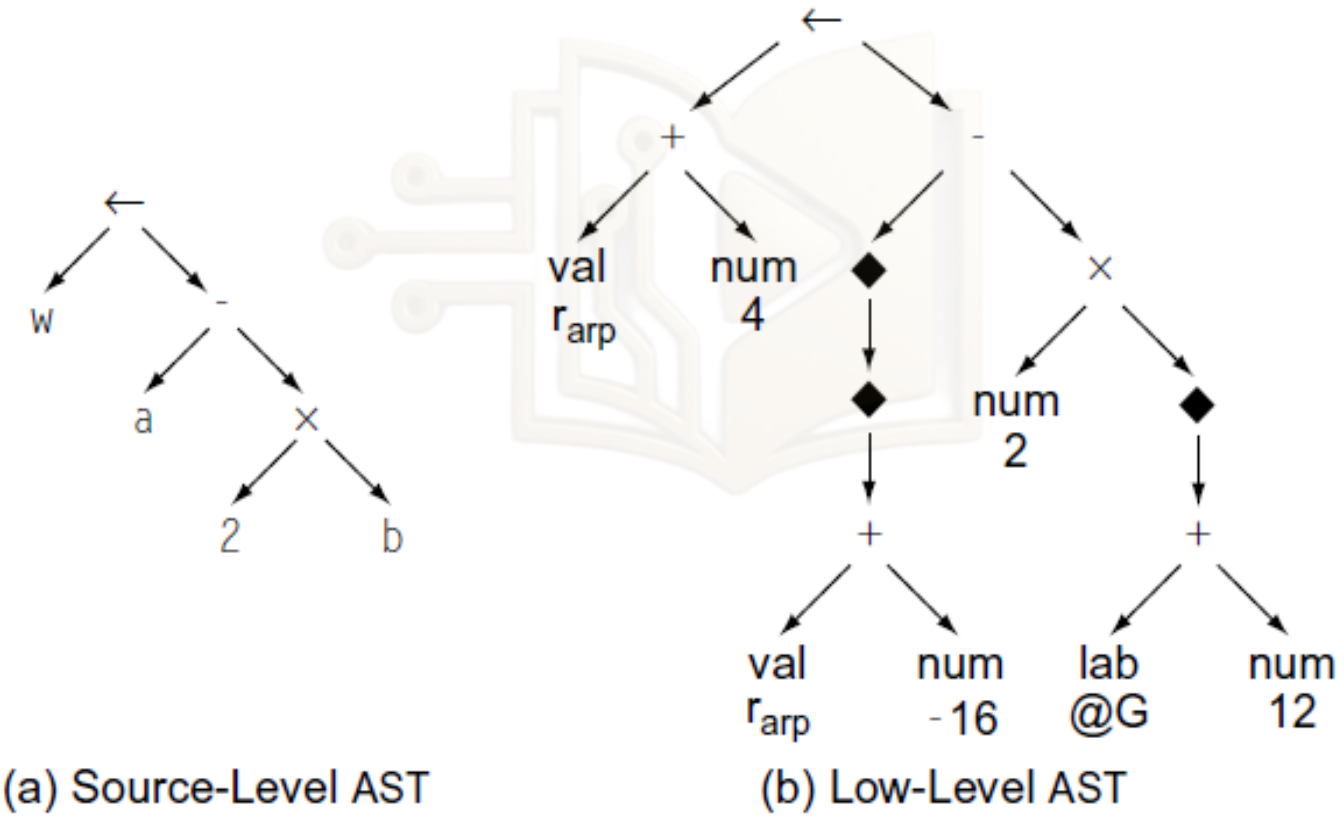


Available on notes.arpanadhikari7.com.np

Level of Abstraction

- Compilers also use low-level trees.
- Tree-based techniques for optimization and code generation, in fact, may require such detail.
- As an example, consider the statement w a - 2 × b.
- A source-level AST creates a concise form, as shown.
- However, the source-level tree lacks much of the detail needed to translate the statement into assembly code.
- A low-level tree, shown below, can make that detail explicit.

Available on notes.arpanadhikari7.com.np



(a) Source-Level AST

(b) Low-Level AST

Available on notes.arpanadhikari7.com.np

- This tree introduces four new node types. A val node represents a value already in a register.
- A num node represents a known constant.
- A lab node represents an assembly-level label, typically a relocatable symbol.
- Finally, □ is an operator that dereferences a value; it treats the value as a memory address and returns the contents of the memory at that address.

Available on notes.arpanadhikari7.com.np

- The low-level tree reveals the address calculations for the three variables.
- w is stored at offset 4 from the pointer in r_{arp}, which holds the pointer to the data area for the current procedure.
- The double dereference of a shows that it is a call-by-reference formal parameter accessed through a pointer stored 16 bytes before r_{arp}.
- Finally, b is stored at offset 12 after the label @G.

Available on notes.arpanadhikari7.com.np

- The level of abstraction matters because the compiler can, in general, only optimize details that are exposed in the IR.
- Properties that are implicit in the IR are hard to change, in part because the compiler would need to translate implicit facts in different, instance-specific ways.
- For example, to customize the code generated for an array reference, the compiler must rewrite the related IR expressions.
- In a real program, different array references are optimized in different ways, each according to the surrounding context.

Available on notes.arpanadhikari7.com.np

Graphs

- While trees provide a natural representation for the grammatical structure of the source code discovered by parsing, their rigid structure makes them less useful for representing other properties of programs.

- To model these aspects of program behavior, compilers often use more general graphs as IRs.

- DAG is an example of graph.

Available on notes.arpanadhikari7.com.np

- ***Control-Flow Graph***
- A Control Flow Graph (CFG) is a fundamental data structure used in compilers to represent the flow of execution through a program.

- It's a directed graph that captures all possible paths that might be traversed during program execution.

- A CFG is a directed graph where:

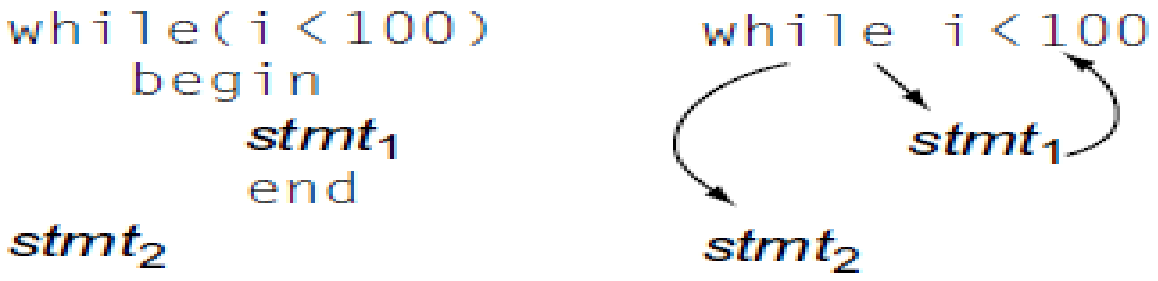
- Nodes represent basic blocks (sequences of instructions with a single entry and exit point)
- Edges represent possible flow of control between basic blocks (jumps, branches, etc.)

Available on notes.arpanadhikari7.com.np

- A *control-flow graph (CGF) models the flow of control between the basic* blocks in a program.
- A CGF is a directed graph, *G =(N, E).*
- *Each node n ∈ N corresponds to a basic block.*
- *Each edge e =(n_i ,n_j) ∈ E corresponds* to a possible transfer of control from block n_i *to block* n_j .

Available on notes.arpanadhikari7.com.np

- The CFG provides a graphical representation of the possible runtime control flow
- paths. The CFG differs from the syntax-oriented IRs, such as an AST, in which the edges show grammatical structure.
- Consider the following CFG for a while loop:
- The edge from *stmt1 back to the loop header creates a cycle; the AST for this* fragment would be acyclic.



Available on notes.arpanadhikari7.com.np

- **Basic Blocks**:
  - Maximal sequence of consecutive instructions with:
    - Single entry point (no jumps into middle of block)
    - Single exit point (no jumps out until last instruction)
  - Ends with a branch or jump instruction
- **Edges**:
  - Represent transitions between blocks
  - Can be conditional (if-then-else) or unconditional (goto)

Available on notes.arpanadhikari7.com.np

Construction Steps

- **Identify leaders** (first instructions of basic blocks):
  - First instruction of program
  - Targets of jumps/branches
  - Instructions following jumps/branches
- **Form basic blocks**:
  - Each leader starts a new block
  - Block continues until next leader or end of program
- **Connect blocks with edges**:
  - Add edge from block A to block B if:
    - B immediately follows A and A doesn't end with unconditional jump
    - A ends with jump to B

Available on notes.arpanadhikari7.com.np

```
1: x = 1
2: y = 2
3: if x < y then
4:    z = x + y
5: else
6:    z = x - y
7: print z
```

Available on notes.arpanadhikari7.com.np

Dependence Graph

- A **dependence graph** (or **dependency graph**) is a compiler intermediate representation (IR) that captures relationships between operations based on data flow and control flow.

- It helps analyze and optimize programs by explicitly showing which operations depend on others.

Available on notes.arpanadhikari7.com.np

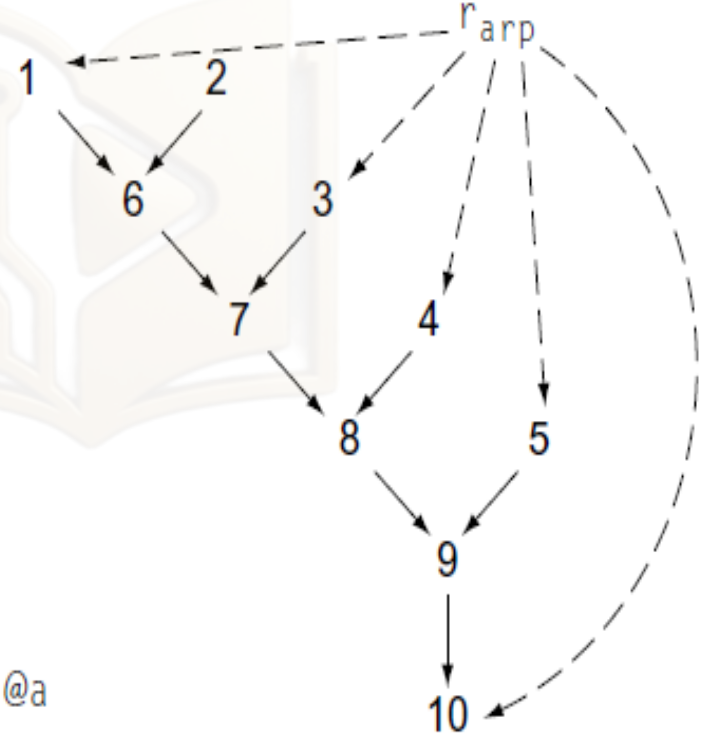
- A data dependence graph embodies this relationship. Nodes in a data-dependence graph represent operations.
- Most operations contain both definitions and uses.
- An edge in a data-dependence graph connects two nodes, one that defines a value and another that uses it.
- We draw dependence graphs with edges that run from definition to use.

Available on notes.arpanadhikari7.com.np

- Lets consider the example of ILOC Code for a a × 2 × b × c × d
- The graph has a node for each statement in the block.
- Each edge shows the flow of a single value.
- For example, the edge from 3 to 7 reflects the definition of r_b in statement 3 and its subsequent use in statement 7.
- r_{arp} contains the starting address of the local data area.
- Uses of r_{arp} refer to its implicit definition at the start of the procedure; they are shown with dashed lines.

Available on notes.arpanadhikari7.com.np

1	loadAI	r_{arp}, @a	⇒	r_a
2	loadI	2	⇒	r_2
3	loadAI	r_{arp}, @b	⇒	r_b
4	loadAI	r_{arp}, @c	⇒	r_c
5	loadAI	r_{arp}, @d	⇒	r_d
6	mult	r_a, r_2	⇒	r_a
7	mult	r_a, r_b	⇒	r_a
8	mult	r_a, r_c	⇒	r_a
9	mult	r_a, r_d	⇒	r_a
10	storeAI	r_a	⇒	r_{arp}, @a



Available on notes.arpanadhikari7.com.np

Call Graph

- A call graph is a fundamental data structure used in compiler design that represents calling relationships between functions or procedures in a program.

- When integrated with intermediate representation (IR), it becomes a powerful tool for program analysis and optimization.

- A call graph is a directed graph where:
- **Nodes** represent functions/procedures
- **Edges** represent calls between functions (an edge from A to B means A calls B)

Available on notes.arpanadhikari7.com.np

- Indeed, constructing accurate call graphs is challenging due to both software engineering practices and modern programming language features. Here are the key complicating factors:

- **Separate compilation**, the practice of compiling small subsets of a program independently, limits the compiler's ability to build a call graph and to perform interprocedural analysis and optimization. Some compilers build partial call graphs for all of the procedures in a compilation unit and perform analysis and optimization across that set. To analyze and optimize the whole program in such a system, the programmer must present it all to the compiler at once.

- Any technique that examines interactions across multiple procedures is called *interprocedural*.

Available on notes.arpanadhikari7.com.np

- Procedure-valued parameters, both as input parameters and as return values, complicate call-graph construction by introducing ambiguous call sites.

- If fee takes a procedure-valued argument and invokes it, that site has the potential to call a different procedure on each invocation of fee.

- The compiler must perform an interprocedural analysis to limit the set of edges that such a call induces in the call graph.

Available on notes.arpanadhikari7.com.np

- **Object-oriented programs** with inheritance routinely create ambiguous procedure calls that can only be resolved with additional type information.
- In some languages, interprocedural analysis of the class hierarchy can provide the information needed to disambiguate these calls.
- In other languages, that information cannot be known until runtime.
- Runtime resolution of ambiguous calls poses a serious problem for call graph construction; it also creates significant runtime overheads on the execution of the ambiguous calls.

Available on notes.arpanadhikari7.com.np

Linear Intermediate Representation

- A **Linear Intermediate Representation (Linear IR)** is a type of **Intermediate Representation (IR)** used in compilers.
- It represents a program as a **linear sequence of instructions**, similar to assembly language, where each instruction is executed one after the other in a straight line, unless a control flow instruction (like a jump or branch) changes that order.
- A linear intermediate code may have one operand address, two-address, or three-address like RISC architectures.
- In fact it may also be zero-address.

Available on notes.arpanadhikari7.com.np

- Many kinds of linear IRs have been used in compilers.
- **One-address codes** model the behavior of accumulator machines and stack machines. These codes expose the machine's use of implicit names so that the compiler can tailor the code for it. The resulting code is quite compact.

- **Two-address codes** model a machine that has destructive operations. These codes fell into disuse as memory constraints became less important.

- Three-address codes model a machine where most operations take two operands and produce a result. The rise of RISC architectures in the 1980s and 1990s made these codes popular, since they resemble a simple RISC machine.

Available on notes.arpanadhikari7.com.np

- The remainder of this section describes two linear IRs that remain popular: stack-machine code and three-address code. Stack-machine code offers a compact, storage-efficient representation.

- Three-address code models the instruction format of a modern RISC machine; it has distinct names for two operands and a result.

Available on notes.arpanadhikari7.com.np

Stack-Machine Code

- Stack-machine code, a form of one-address code, assumes the presence of a stack of operands.
- Most operations take their operands from the stack and push their results back onto the stack.
- For example, an integer sub tract operation would remove the top two elements from the stack and push their difference onto the stack.

Available on notes.arpanadhikari7.com.np

- Stack-machine code is compact. The stack creates an implicit name space and eliminates many names from the IR.
- This shrinks the size of a program in IR form.
- Using the stack, however, means that all results and arguments are transitory, unless the code explicitly moves them to memory.
- Stack-machine code for the expression a - 2 × b

```
push 2
push b
multiply
push a
subtract
```

Available on notes.arpanadhikari7.com.np

Advantages

- compact form
- introduced names are implicit, not explicit
- simple to generate & execute code

Disadvantages

- does not match current architectures
- many spurious *dependences* due to stack:
  $\Rightarrow$ difficult to do reordering transformations
- cannot "reuse" expressions easily (must store and re-load)
  $\Rightarrow$ difficult to express optimized code

Available on notes.arpanadhikari7.com.np

Three-Address Code

- Three-address code (TAC) is an intermediate representation used by compilers where each instruction has at most three operands.
- It breaks down complex expressions and control flow into simple instructions, where **each instruction contains at most three addresses** (typically two operands and one result).
- Three-address code is attractive for several reasons.
- First, three-address code is reasonably compact. Most operations consist of four items: an operation and three names.
- Both the operation and the names are drawn from limited sets.

Available on notes.arpanadhikari7.com.np

- Operations typically require 1 or 2 bytes.
- Names are typically represented by integers or table indices; in either case, 4 bytes is usually enough.
- Second, separate names for the operands and the target give the compiler freedom to control the reuse of names and values.
- Finally, since many modern processors implement three-address operations, a three-address code models their properties well.

Available on notes.arpanadhikari7.com.np

Three address code for a - 2 × b

t1 ← 2

t2 ← b

t3 ← t1 × t2

t4 ← a

t5 ← t4 - t3

Example

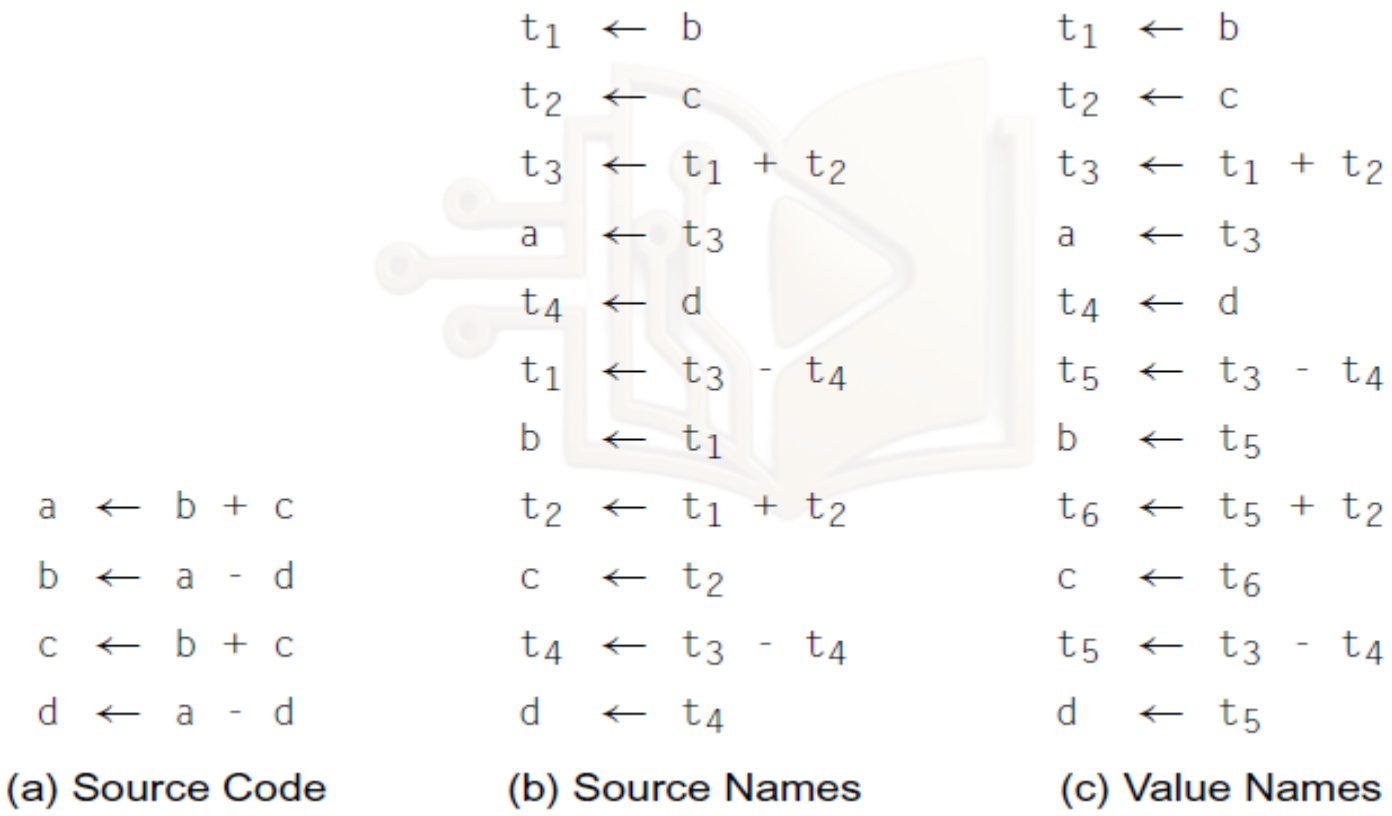
z = (a + b) * (c - d) / e	if (x < y) {
	  z = x + y;
	} else {
	  z = x - y;
	}
		if (x > y) z=x-2 * y

Available on notes.arpanadhikari7.com.np

Mapping values to names

- Temporary values (or "temps") are intermediate results generated during program execution or compilation.
- Properly naming them is crucial for readability, optimization, and correctness in compiler intermediate representations (IRs).
- The IR form of a program usually contains more detail than does the source version.
- Some of those details are implicit in the source code; others result from deliberate choices in the translation.
- To see this, consider the four-line block of source code shown in Figure a below.

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

- The block deals with just four names, { a, b, c, d }. It refers to more than four values.
- Each of b, c, and d have a value before the first statement executes.
- The first statement computes a new value, b + c, as does the second, which computes a - d.
- The expression b + c in the third statement computes a different value than the earlier b + c, unless c = d initially.
- Finally, the last statement computes a - d; its result is always identical to that produced by the second statement.

Available on notes.arpanadhikari7.com.np

- The source code names tell the compiler little about the values that they hold.
- For example, the use of b in the first and third statements refer to distinct values (unless c = d).
- The reuse of the name b conveys no information; in fact, it might mislead a casual reader into thinking that the code sets a and c to the same value.
- When the compiler names each of these expressions, it can chose names in ways that specifically encode useful information about their values.

Available on notes.arpanadhikari7.com.np

- Consider, for example, the translations shown in Figures b and c.
- These two variants were generated with different naming disciplines.
- The code in Figure b uses fewer names than the code in c.
- It follows the source code names, so that a reader can easily relate the code back to the code in Figure a.
- The code in Figure c uses more names than the code in b.

Available on notes.arpanadhikari7.com.np

- Its naming discipline reflects the computed values and ensures that textually identical expressions produce the same result.

- This scheme makes it obvious that a and c may receive different values, while b and d must receive the same value.

Available on notes.arpanadhikari7.com.np

Symbol Table

- The **symbol table** is a critical data structure used to store information about the **identifiers** (such as variable names, function names, classes, objects, etc.) used in a program.
- It acts like a **dictionary** or **map**, where each identifier is associated with attributes such as its type, scope, memory location, and other metadata.
- A **symbol table** is a data structure maintained by the compiler that **associates each identifier in a program with its related information**, such as type, scope, and location.
- It serves as a central repository for information needed during different phases of compilation.

Available on notes.arpanadhikari7.com.np

Each entry in symbol table includes

- **Name** (Identifier)
- **Data type** (int, float, etc.)
- **Scope** (global, local, function-specific)
- **Memory location / offset**
- **Line number(s)** (where used or declared)
- **Kind of identifier** (variable, function, parameter, etc.)
- Dimensions for arrays
- Parameter information for functions
- Return type for functions
- Access modifiers (public, private, etc.)

Available on notes.arpanadhikari7.com.np

Common Operations

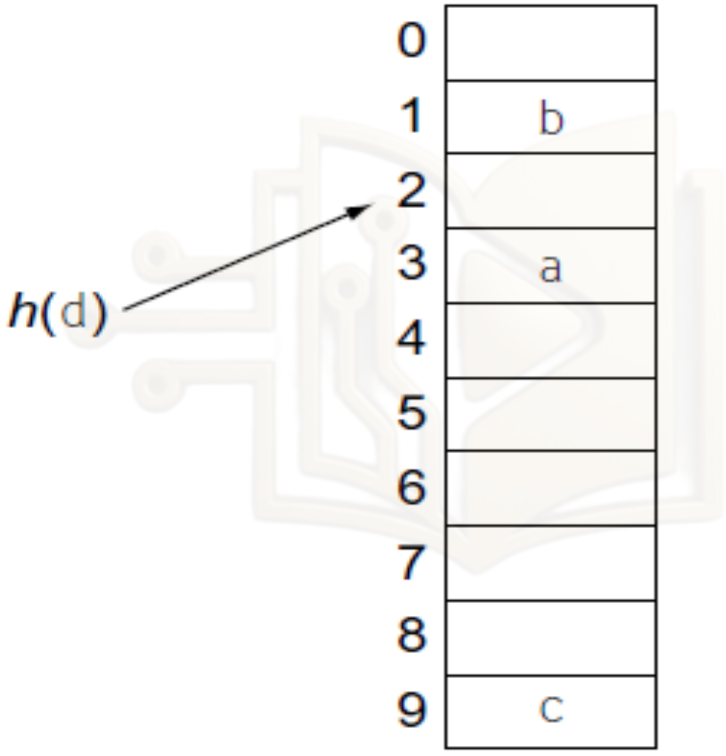
- **Insert**: Add a new symbol and its attributes
- **Lookup**: Find information about a symbol
- **Delete**: Remove symbols when their scope ends
- **Update**: Modify attributes when more information becomes available

Available on notes.arpanadhikari7.com.np

Hash Tables

- A compiler accesses its symbol table frequently.
- Thus, efficiency is a key issue in the design of a symbol table.
- Because hash tables provide constant time expected-case lookups, they are the method of choice for implementing symbol tables.
- Hash tables are conceptually elegant.
- They use a hash function, h, to map names to small integers, and use the small integer to index the table.
- With a hashed symbol table, the compiler stores all the information that it derives about the name n in the table in slot h(n).

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

- The primary reason to use hash tables is to provide a constant-time expected case lookup keyed by a textual name.
- To achieve this, h must be inexpensive to compute.
- Given an appropriate function h, accessing the record for n requires computing h(n) and indexing into the table at h(n).
- If h maps two or more symbols to the same small integer, a "collision" occurs. (In the marginal figure, this would occur if h(d)= 3.)
- The implementation must handle this situation gracefully, preserving both the information and the lookup time.
- A well-implemented hash table can provide fast insertion and a fast test for the presence of a specific edge.

Available on notes.arpanadhikari7.com.np

Building a System Table

- The symbol table defines two interface routines for the rest of the compiler.

1. LookUp(name) returns the record stored in the table at *h(name) if one* exists.

- Otherwise, it returns a value indicating that name was not found.

2. Insert(name,record) stores the information in record in the table at *h(name)*.

- *It may expand the table to accommodate the record for name.*

Available on notes.arpanadhikari7.com.np

- The compiler can use separate functions for LookUp and Insert, or they can be combined by passing LookUp a flag that specifies whether or not to insert the name.
- This ensures, for example, that a LookUp of an undeclared variable will fail.
- In processing declaration syntax, the compiler builds up a set of attributes for each variable.

Available on notes.arpanadhikari7.com.np

- When the parser recognizes a production that declares some variable, it can enter the name and attributes into the symbol table using Insert.

- If a variable name can appear in only one declaration, the parser can call LookUp first to detect a repeated use of the name.

Available on notes.arpanadhikari7.com.np

- When the parser encounters a variable name outside the declaration syntax, it uses LookUp to obtain the appropriate information from the symbol table.
- LookUp fails on any undeclared name.
- The compiler writer, of course, may need to add functions to initialize the table, to store it to and retrieve it from external media, and to finalize it.
- For a language with a single name space, this interface suffices.

Available on notes.arpanadhikari7.com.np